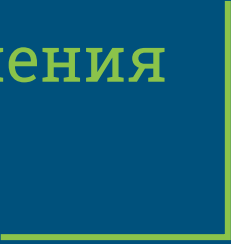




Современные архитектуры ИС



Лекция 5
Основы распределенного обучения



План лекции

1. **Связь нескольких GPU**
2. Коммуникации
3. Параллелизм данных (DP)
4. Тензорный параллелизм (TP)
5. Контекстный параллелизм (CP)
6. Конвейерный параллелизм (PP)
7. 5D параллелизм

Зачем нужны коммуникации?

В одном GPU мы:

- ограничены по памяти
- ограничены скоростью вычислений

Допустим, для предобучения модели нам потребуется $4.8 \cdot 10^{26}$ FLOPs

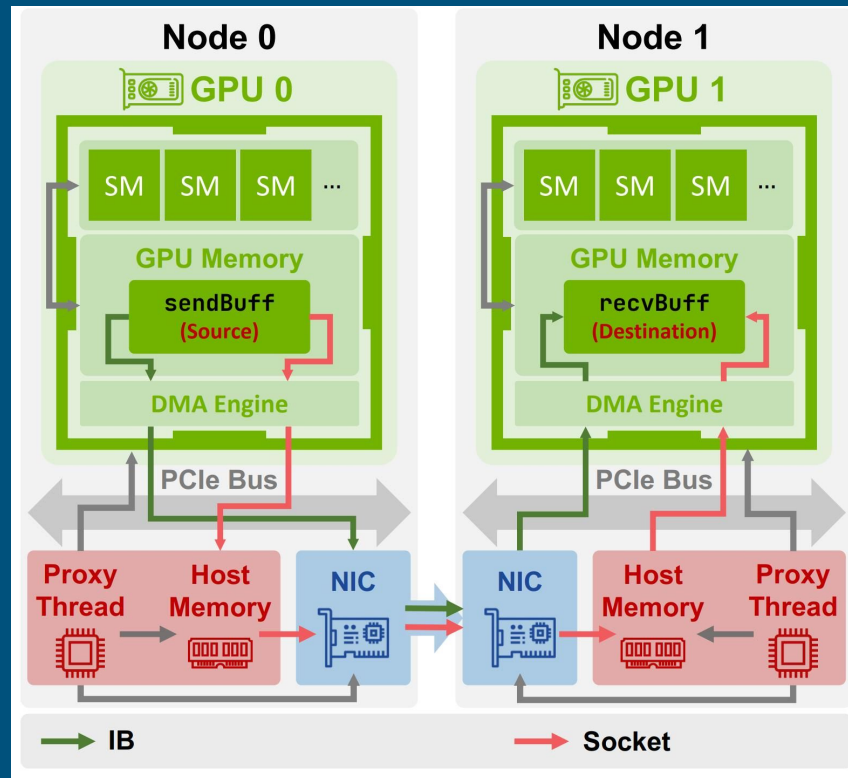
На одной H100 в fp16 типе мы будем обучать на пиковой мощности ~7600 лет.

Устройство GPU-ноды

В ноде обычно находится 8 GPU.

Она состоит из:

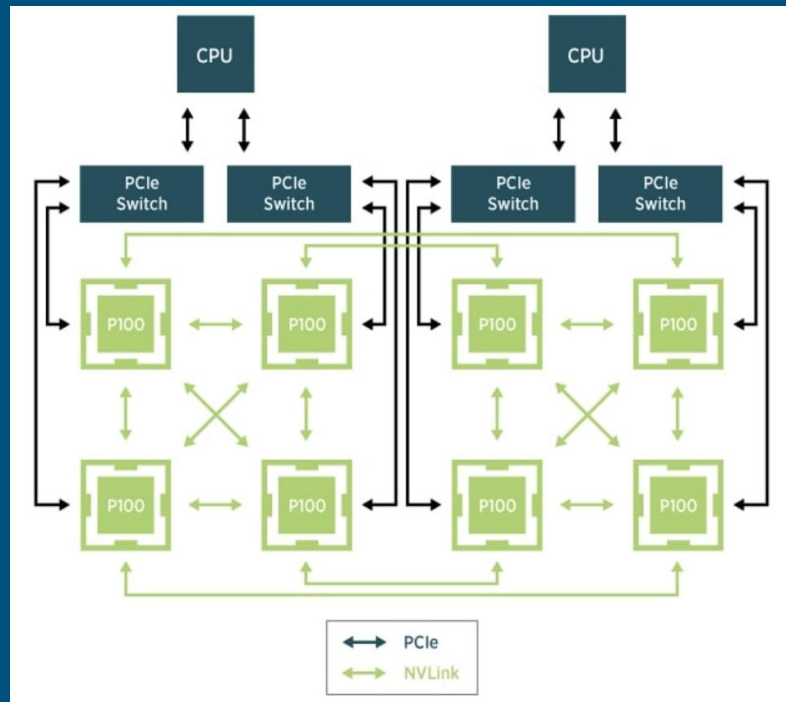
- GPU
- CPU сокетов
- Памяти хоста
- PCIe моста
- Сетевых карт



Внутринодовая связь GPU

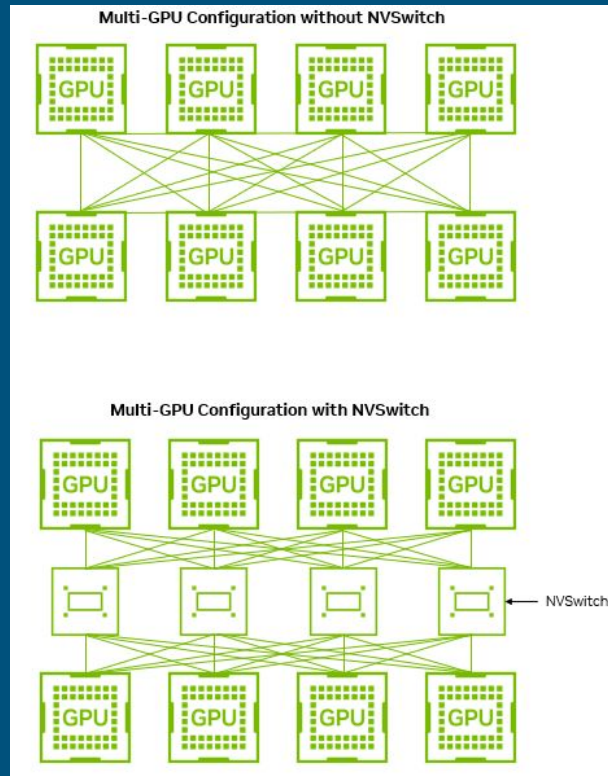
Базовый способ связи — PCIe-шина
(медленно)

Продвинутый способ связи — NVLink
(быстро)

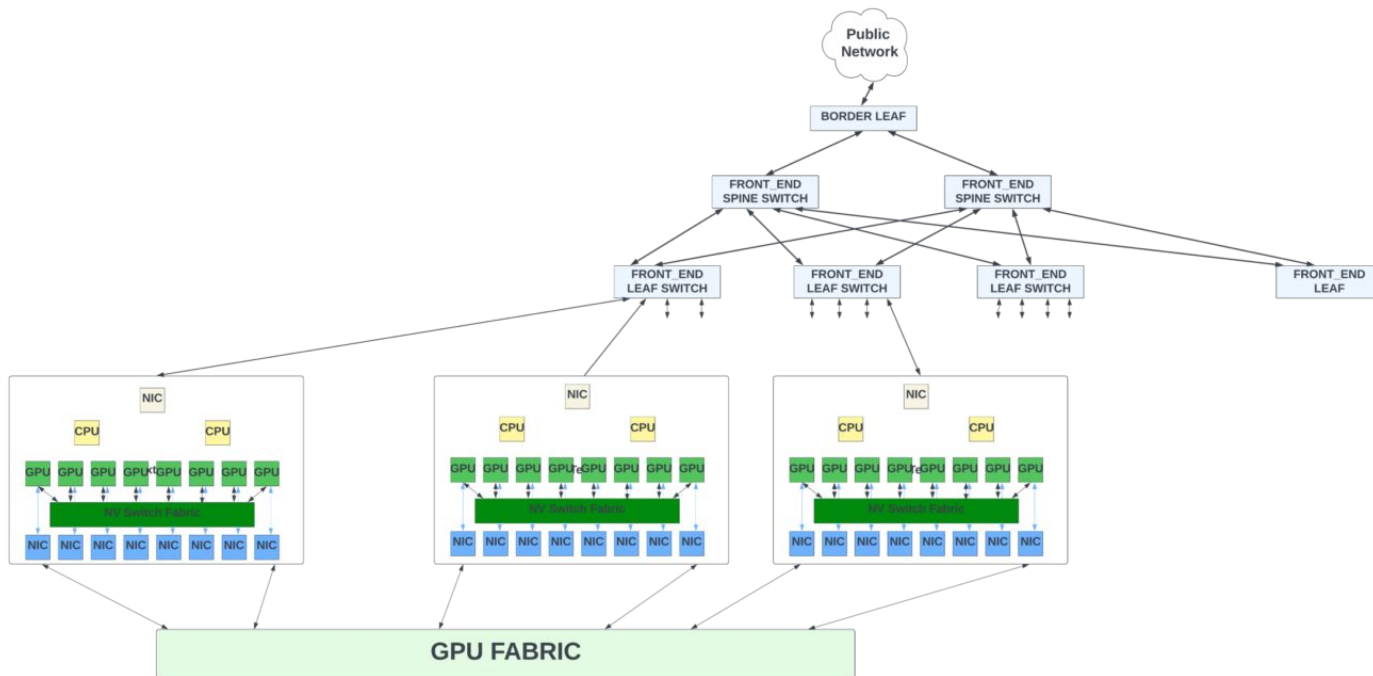


Внутринодовая связь GPU

Чтобы все GPU общались напрямую, и мы могли избежать связи “все-со-всеми”, используется NVSwitch



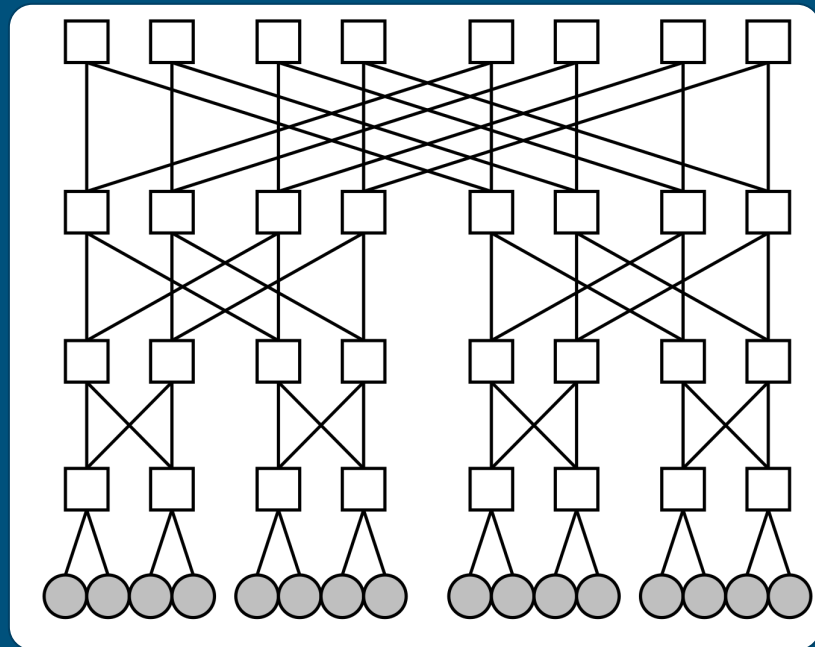
Межнодовая связь GPU



Топология кластера

Объединять тысячи нод по схеме “все-со-всеми” – слишком дорого. Поэтому существуют различные топологии кластеров

- fat-tree (на картинке)
- dragonfly
- torus



Стоимость коммуникаций

Основные метрики, которые оценивают стоимость коммуникаций — это задержка передачи и пропускная способность.

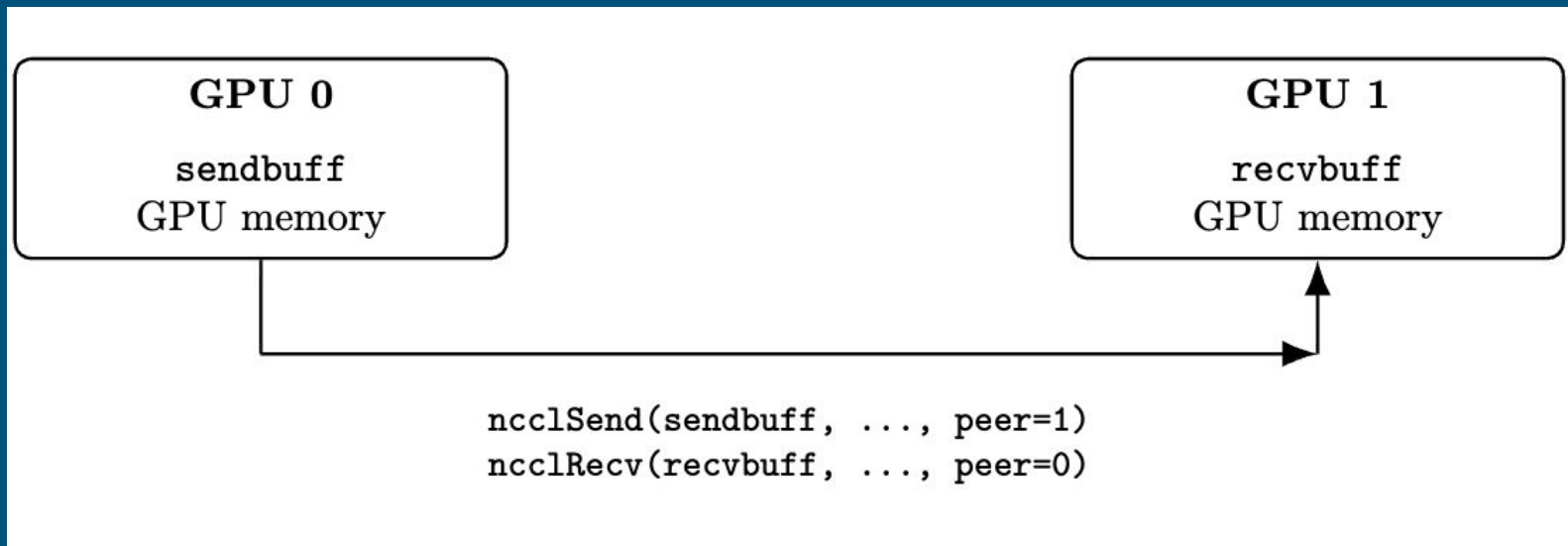
Общая модель стоимости коммуникации:

$$T_{\text{comm}}(n) \approx \alpha + \frac{n}{B}$$

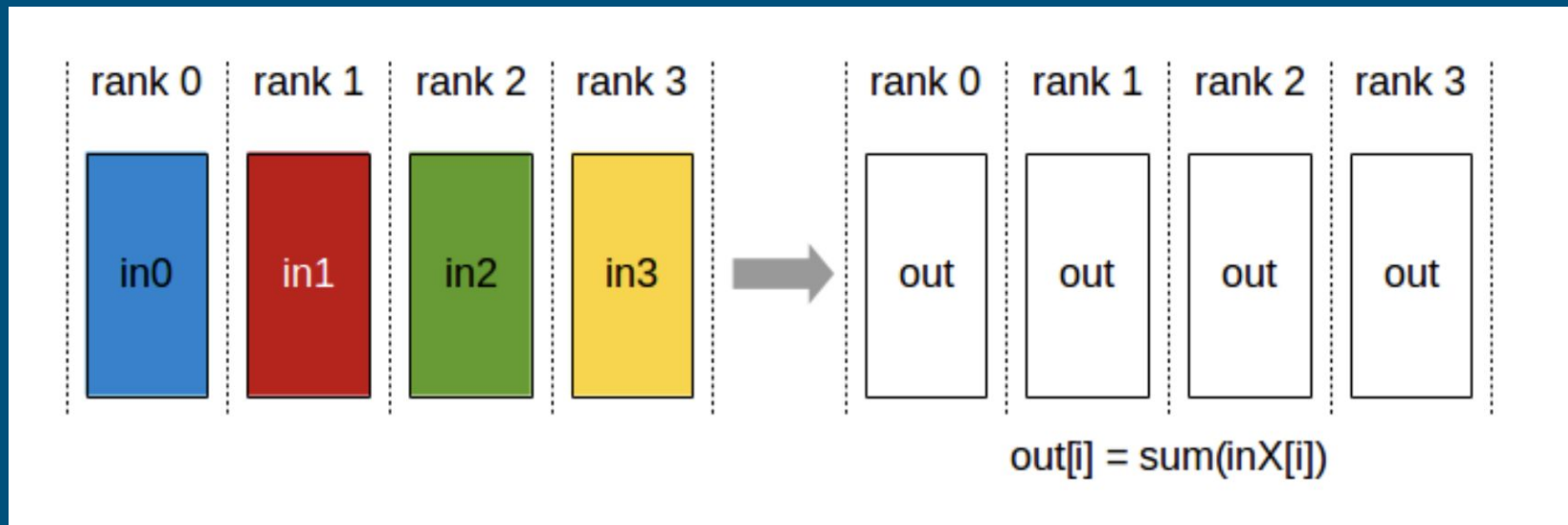
План лекции

1. Связь нескольких
2. **Коммуникации**
3. Параллелизм данных (DP)
4. Тензорный параллелизм (TP)
5. Контекстный параллелизм (CP)
6. Конвейерный параллелизм (PP)
7. 5D параллелизм

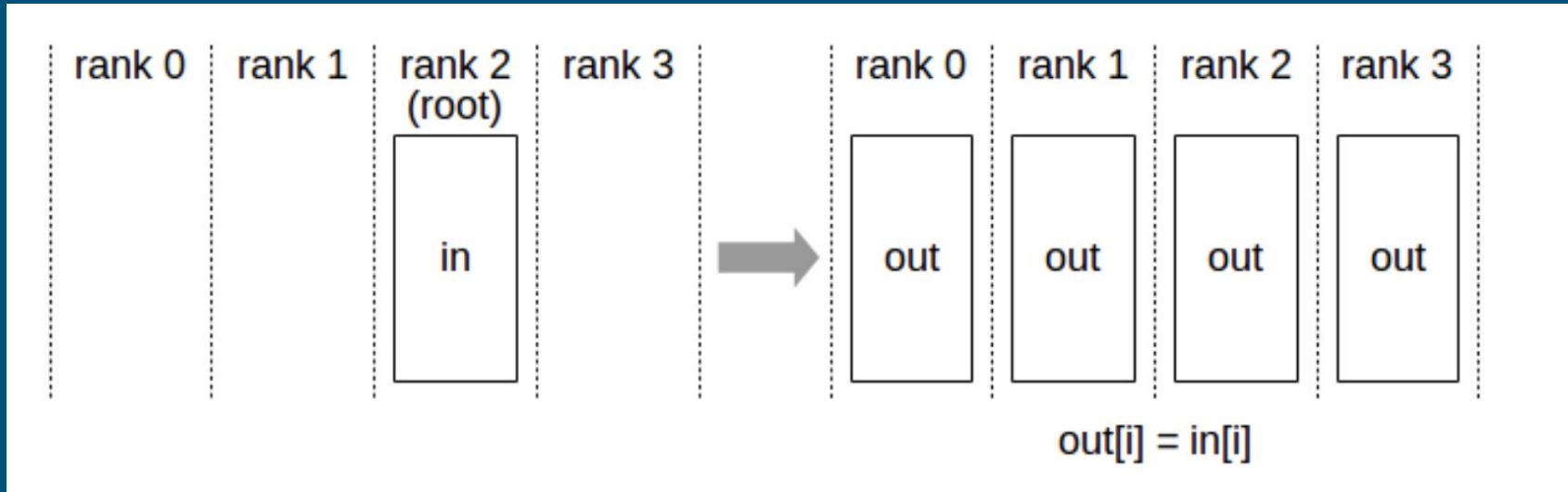
Point-to-point коммуникации



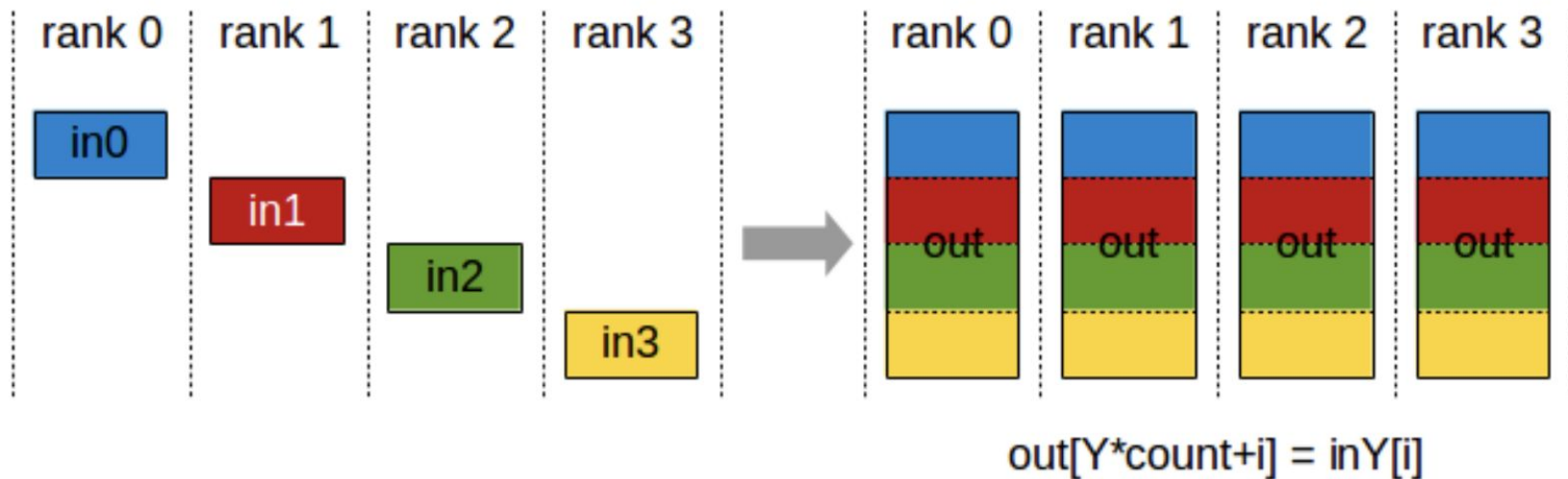
Коллективные коммуникации. AllReduce



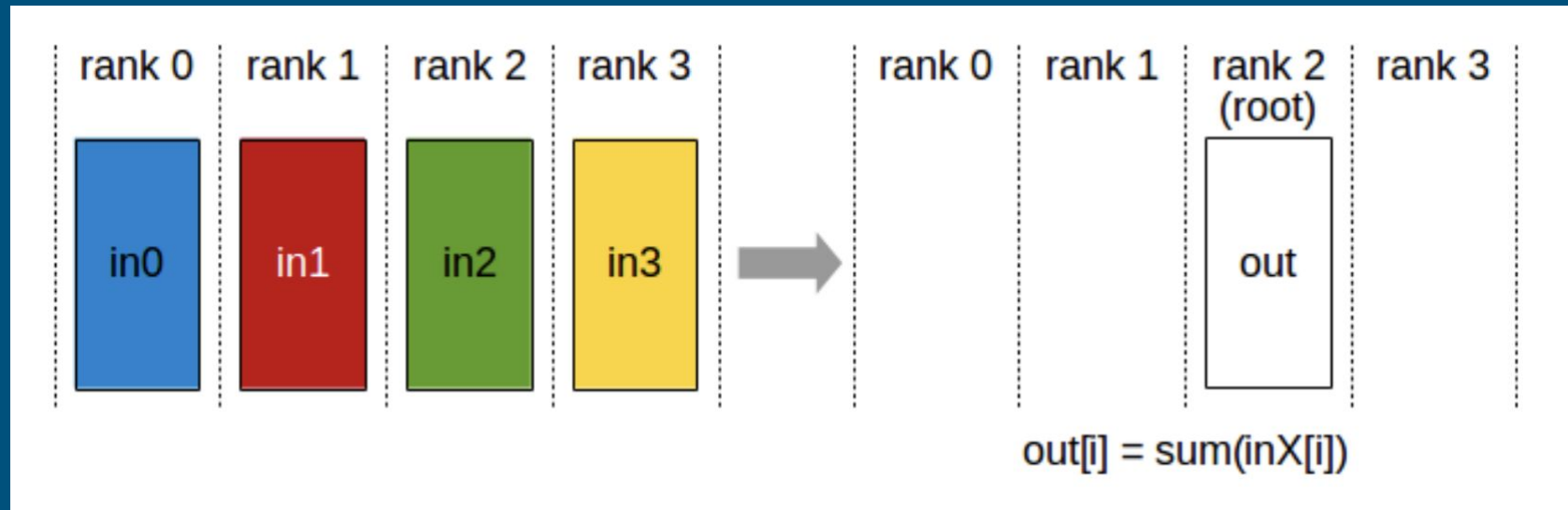
Коллективные коммуникации. Broadcast



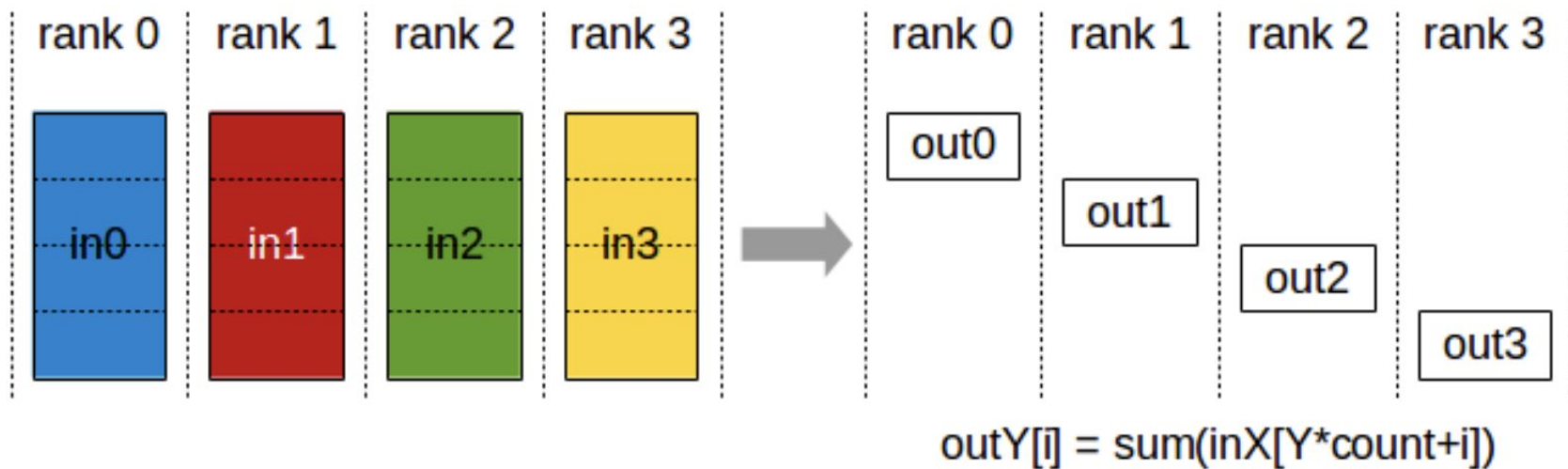
Коллективные коммуникации. AllGather



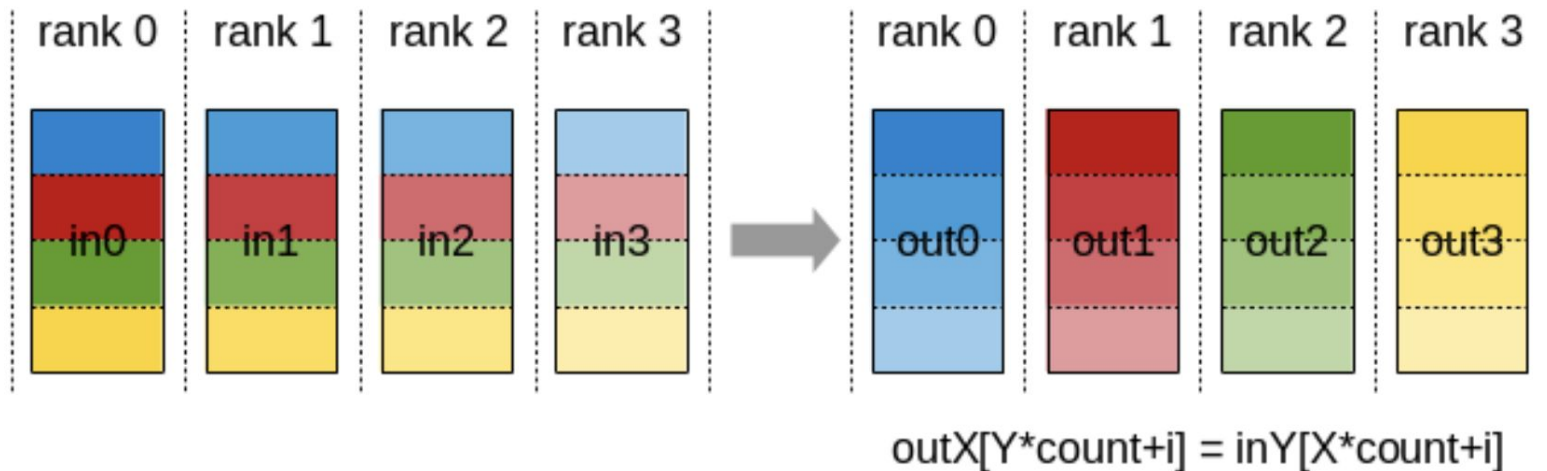
Коллективные коммуникации. Reduce



Коллективные коммуникации. ReduceScatter



Коллективные коммуникации. All2All



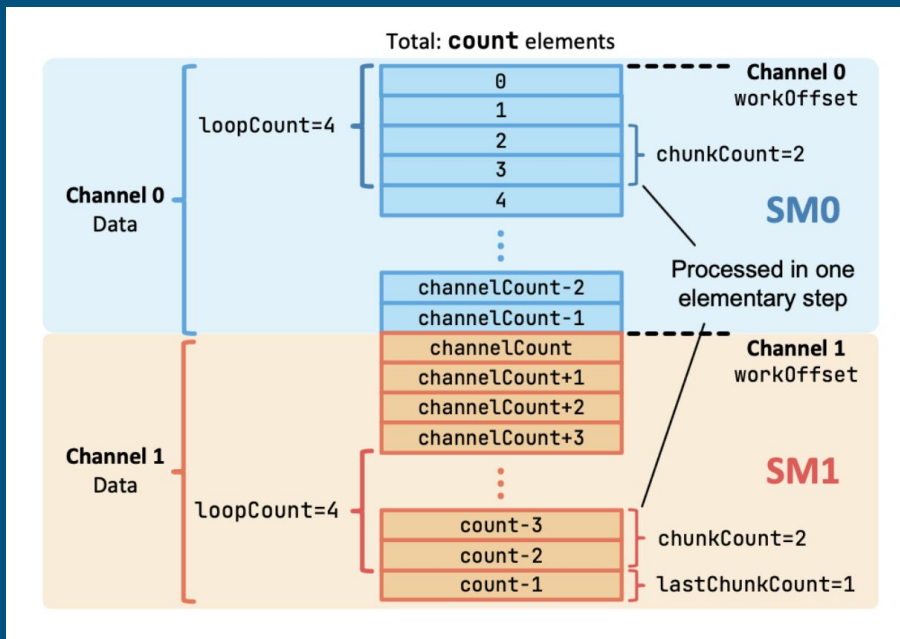
NCCL. Модель исполнения

Основные сущности: ранг, коммуникатор, стрим

Коммуникатор задает группу рангов. Ранги существуют локальные (внутри ноды) и глобальные

Коммуникации обычно располагают в соседнем стриме для асинхронного вычисления

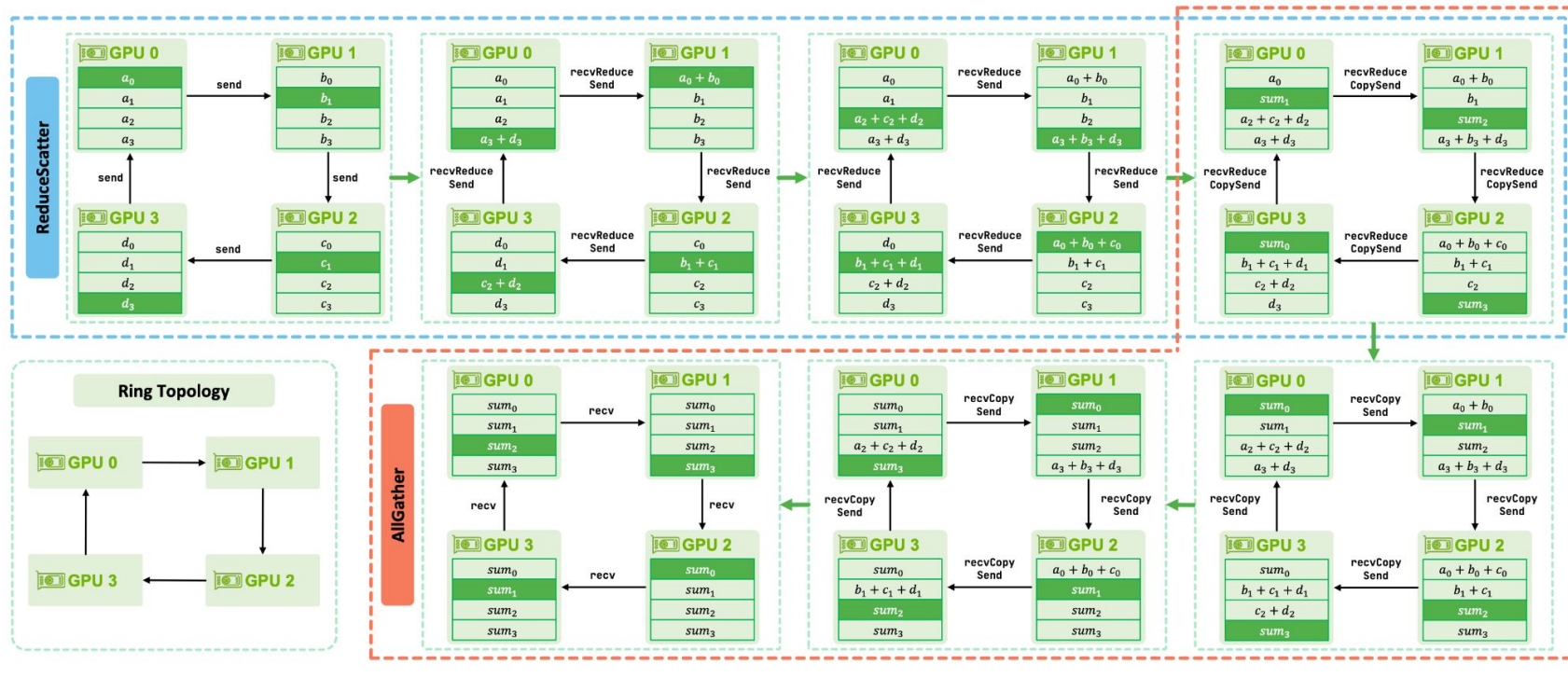
NCCL. Коммуникационные каналы



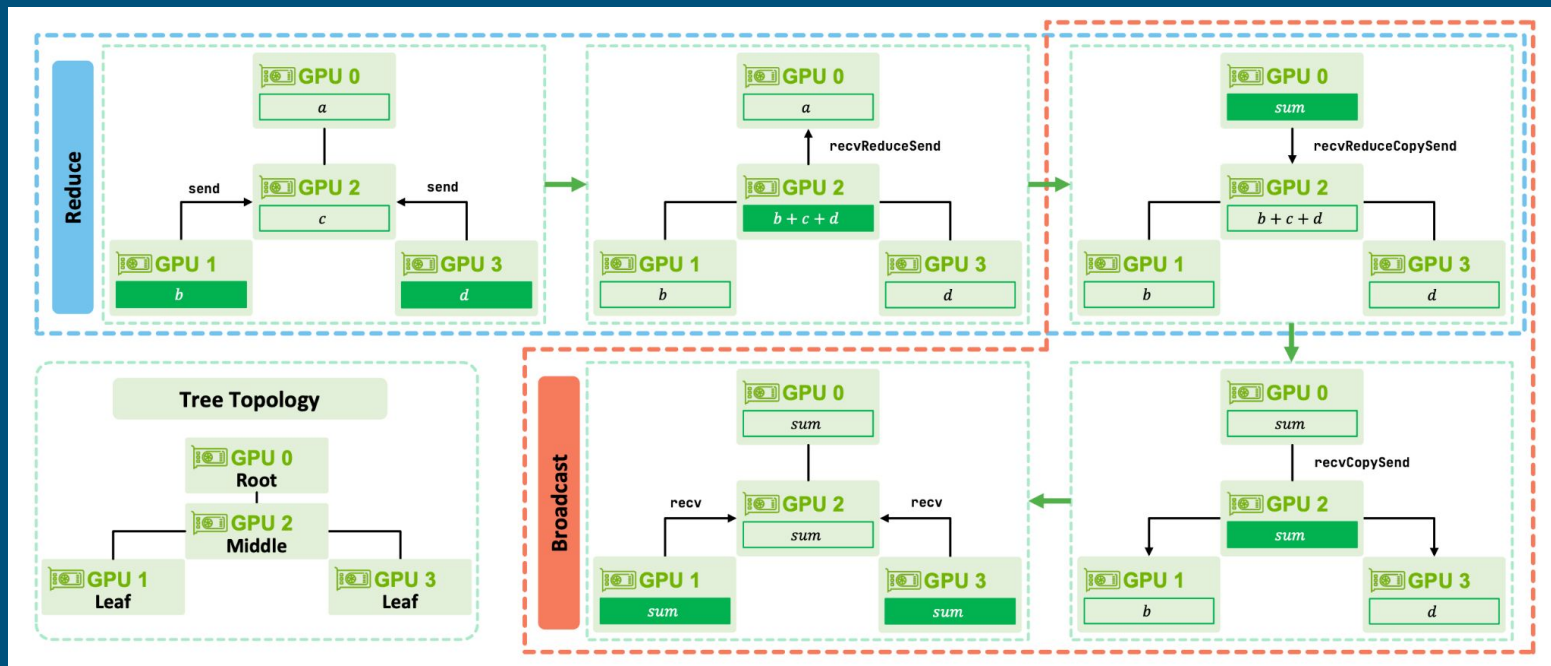
NCCL. Протоколы передачи данных

- Simple
- LL
- LL128

NCCL. Алгоритм “КОЛЬЦО”



NCCL. Алгоритм “дерево”



NCCL. Иерархический алгоритм

Иерархический подход объединяет кольцевой алгоритм и древесный алгоритм

Внутри ноды связь быстрая, можем использовать кольцевой алгоритм

Вне ноды – медленнее, используем древесный алгоритм

План лекции

1. Связь нескольких
2. Коммуникации
- 3. Параллелизм данных (DP)**
4. Тензорный параллелизм (TP)
5. Контекстный параллелизм (CP)
6. Конвейерный параллелизм (PP)
7. 5D параллелизм

Распределенный параллелизм данных (DDP)

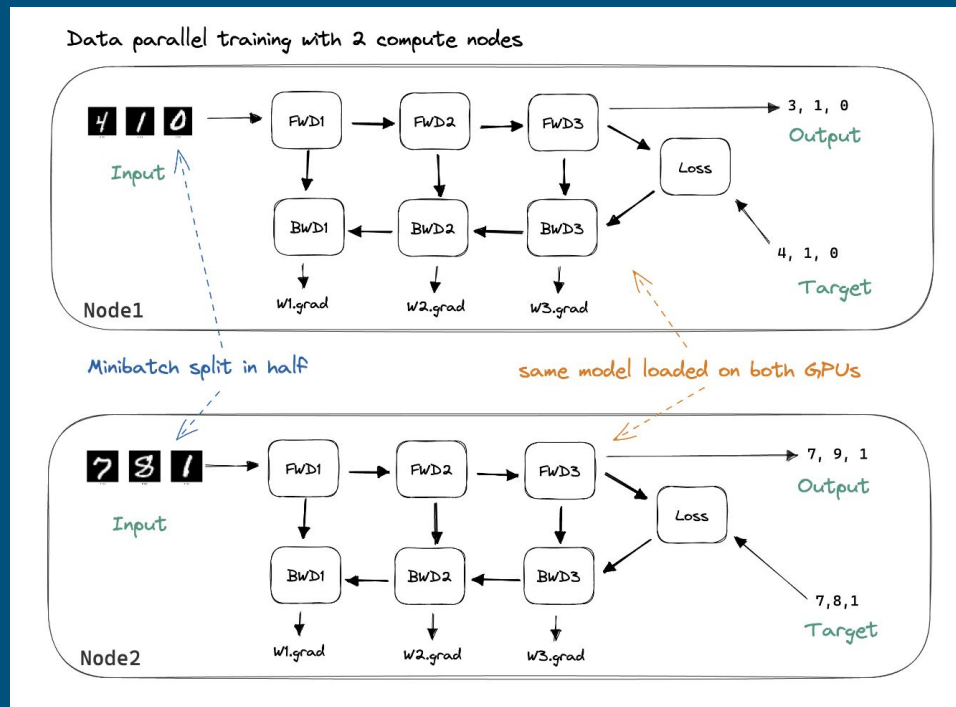
На каждом ранге выполняется
вычисление локального лосса:

$$\mathcal{L}_i^{(t)} = \mathcal{L}_{w_t}(B_i)$$

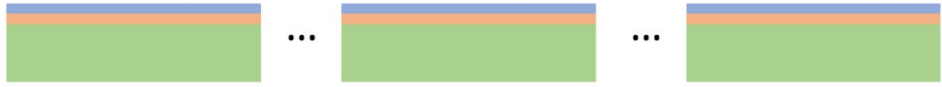
Градиенты будут разные, а веса
одни. Используем AllReduce:

$$g_i^{(t)} = \nabla_{w_t} \mathcal{L}_i^{(t)}$$

$$g^{(t)} = \frac{1}{N} \sum_{j=1}^N g_j^{(t)}$$



ZeRO-1

	gpu ₀ ... gpu _i ... gpu _{N-1}	Memory Consumed	K=12 $\Psi=7.5\text{B}$ $N_d=64$
Baseline		$(2 + 2 + K) * \Psi$	120GB
P _{os}		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB
P _{os+g}		$2\Psi + \frac{(2 + K) * \Psi}{N_d}$	16.6GB
P _{os+g+p}		$\frac{(2 + 2 + K) * \Psi}{N_d}$	1.9GB

■ Parameters
 ■ Gradients
 ■ Optimizer States

ZeRO-2

Шардируем не только состояния оптимизатора, но и градиенты.

Каждый ранг хранит лишь шард градиентов

$$g_i^{(t)} = \nabla_{w_t} \mathcal{L}_i^{(t)}$$

$$g_i^{(t)} = [g_{i1}^{(t)}, g_{i2}^{(t)}, \dots, g_{iN}^{(t)}]$$

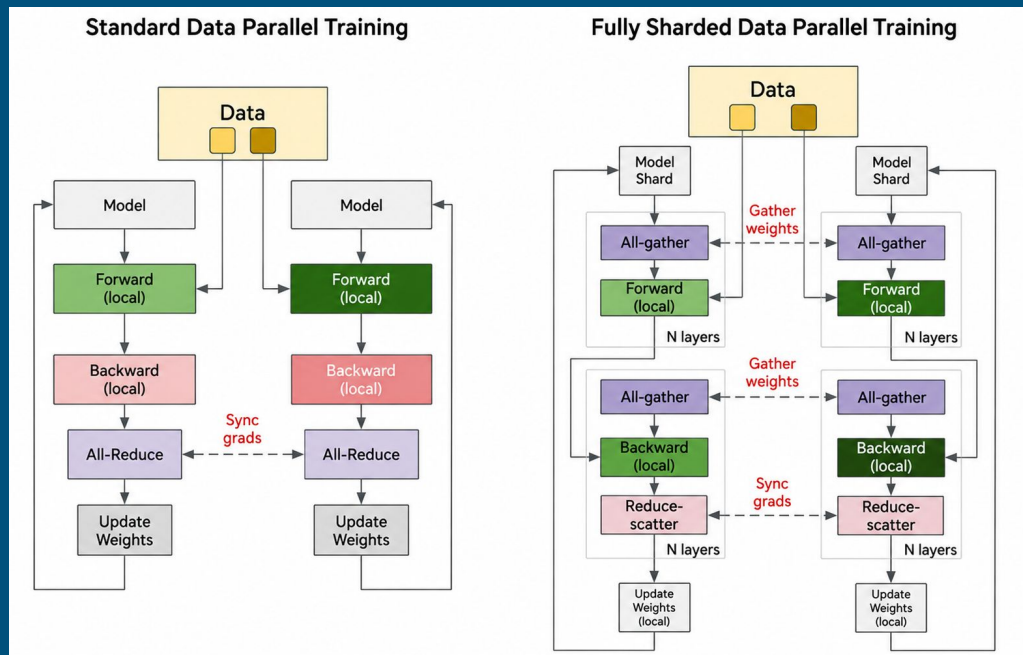
Вместо AllReduce используем Reduce Scatter. Так каждый ранг получит все локальные градиенты, соответствующие его шардированной части, и выполнит редукцию по ней

$$g_i^{(t)} = \frac{1}{N} \sum_{j=1}^N g_{ij}^{(t)}$$

FSDP

Шардируем всё.

Очень большой выигрыш по памяти, но плата — большое количество тяжелых коммуникаций



План лекции

1. Связь нескольких
2. Коммуникации
3. Параллелизм данных (DP)
4. **Тензорный параллелизм (TP)**
5. Контекстный параллелизм (CP)
6. Конвейерный параллелизм (PP)
7. 5D параллелизм

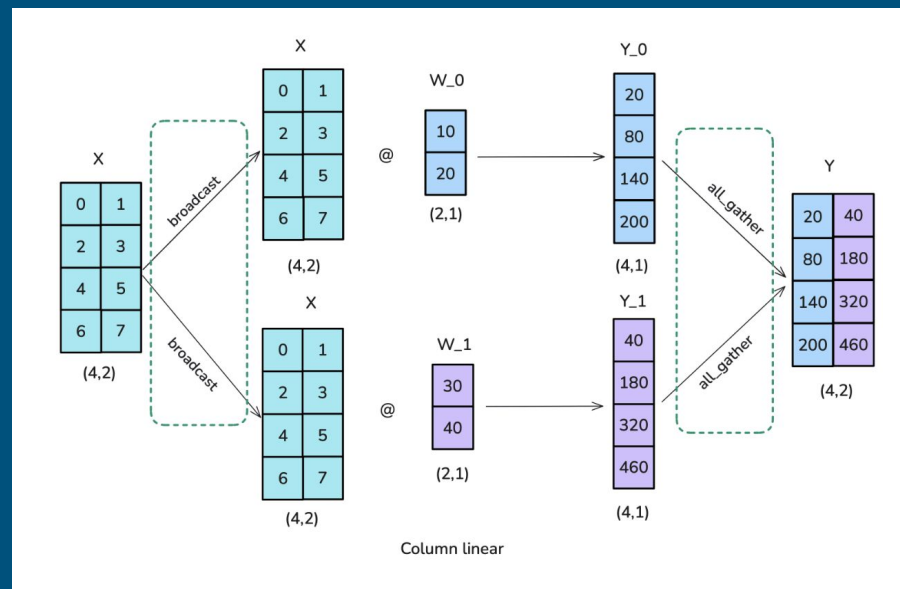
Параллелизм по столбцам

Есть свойство матричного
умножения

$$A \cdot B = A \cdot [B_1 \ B_2 \ \dots] = [A \cdot B_1 \ A \cdot B_2 \ \dots]$$

В линейном слое разобьем матрицу
весов по столбцам

$$Y_i = XW_i$$

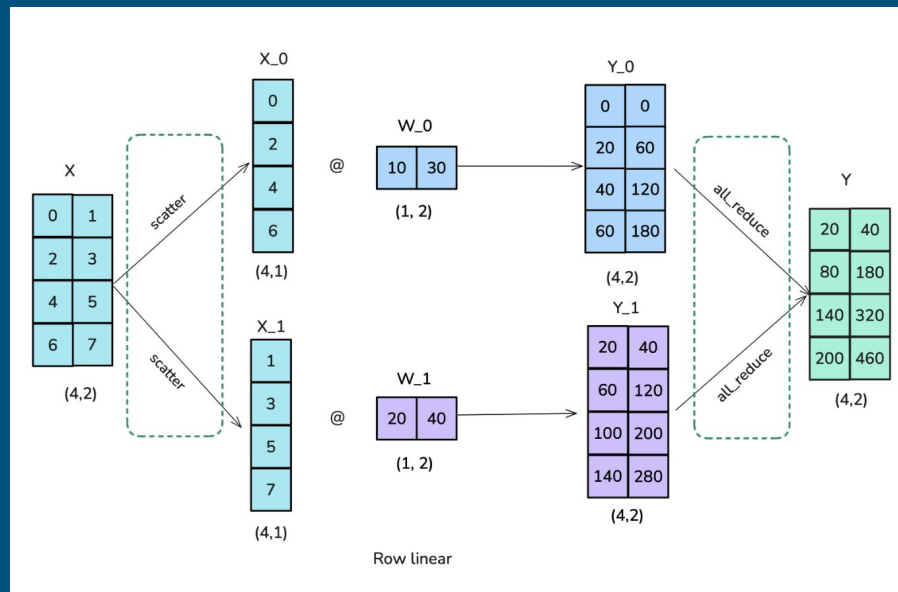


Параллелизм по строкам

Второе интересное для нас свойство

$$A \cdot B = \begin{bmatrix} A_1 & A_2 & \dots \end{bmatrix} \cdot \begin{bmatrix} B_1 \\ B_2 \\ \dots \end{bmatrix} = \sum_{i=1} A_i B_i$$

Это значит, что мы можем матрицу входов между девайсами (и тем самым уменьшить объем хранимых активаций)

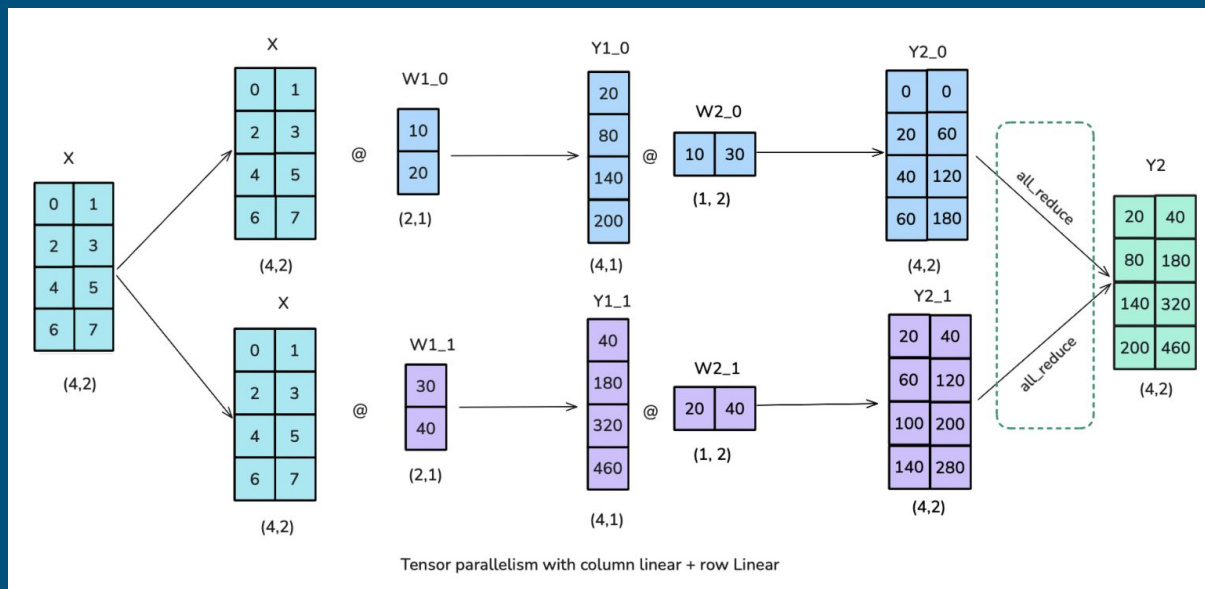


TP в FFN-блоке

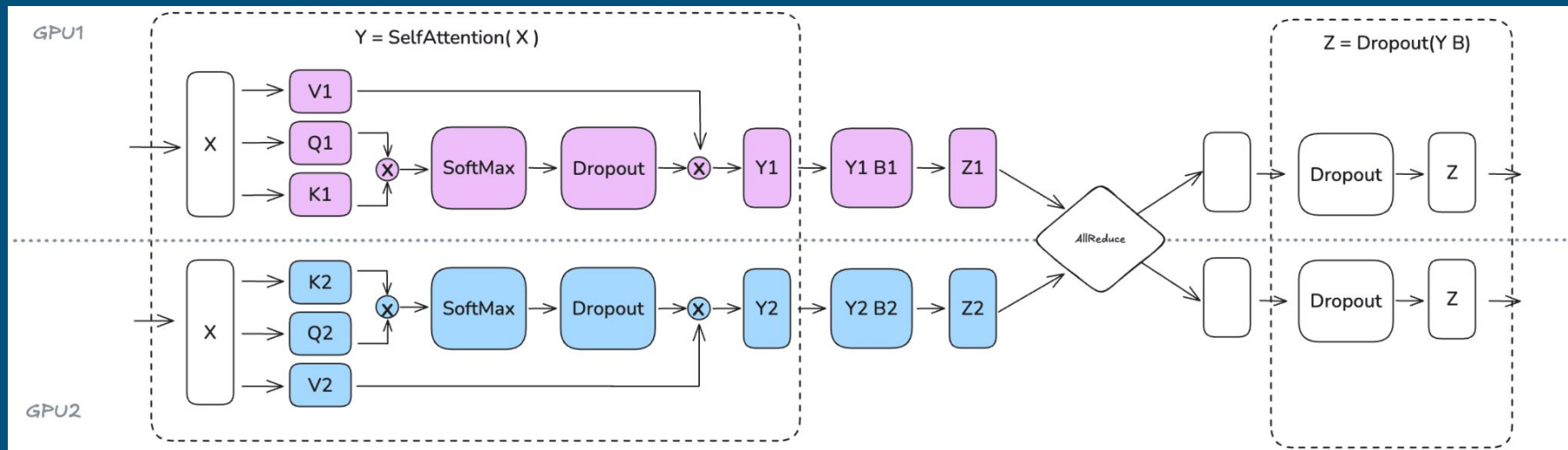
В FFN блоке два линейных слоя.

Применим для первого параллелизм по столбцам

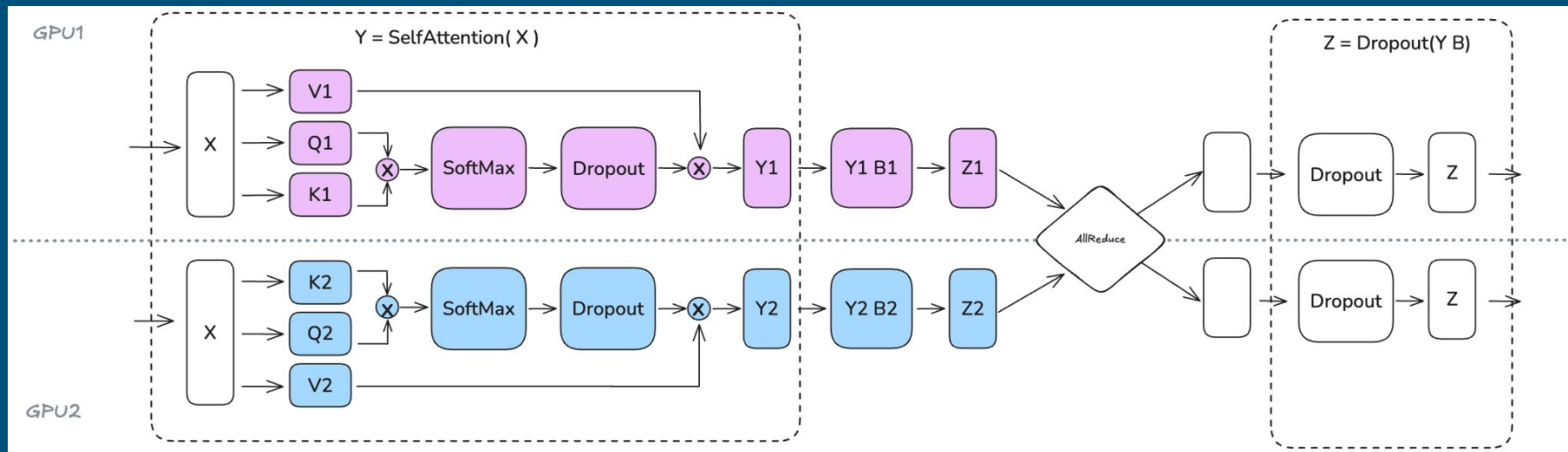
А для второго – параллелизм по строкам



ТР в механизме внимания



ТР в механизме внимания

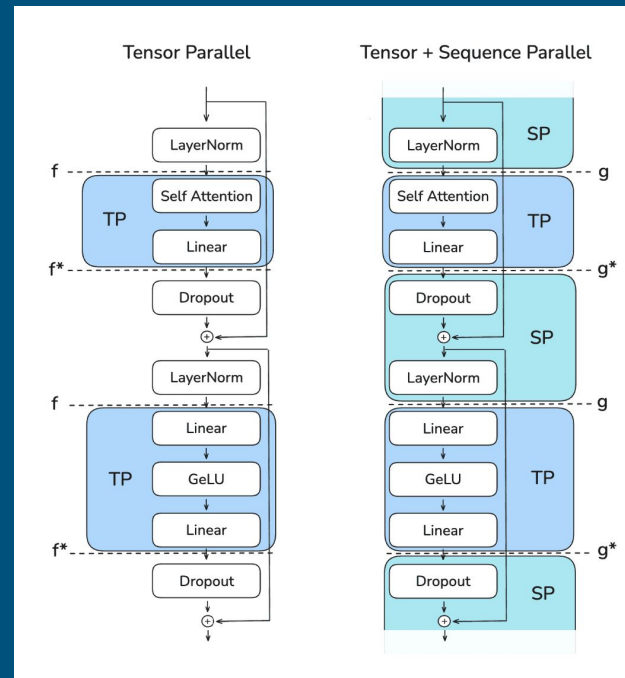


А что с Flash Attention?

Параллелизм последовательностей (SP)

Давайте делить последовательность в нормализациях и в дропаутах на разные девайсы.

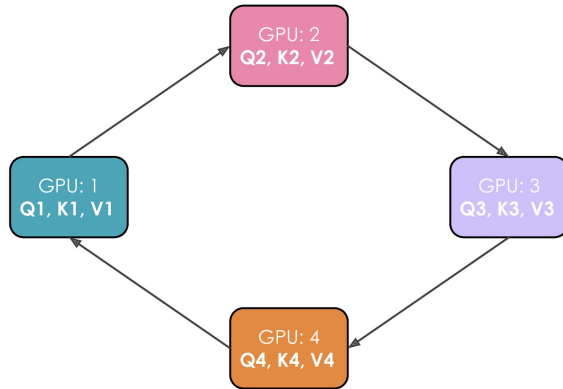
И объединим это в один пайплайн с тензорным параллелизмом



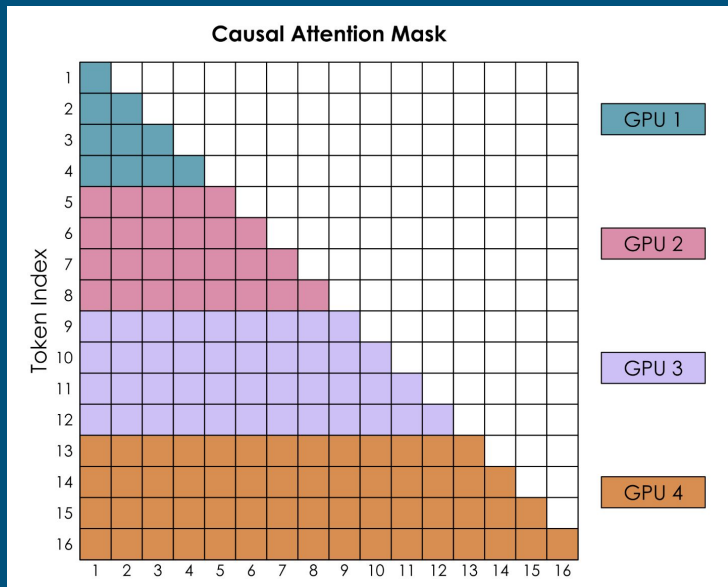
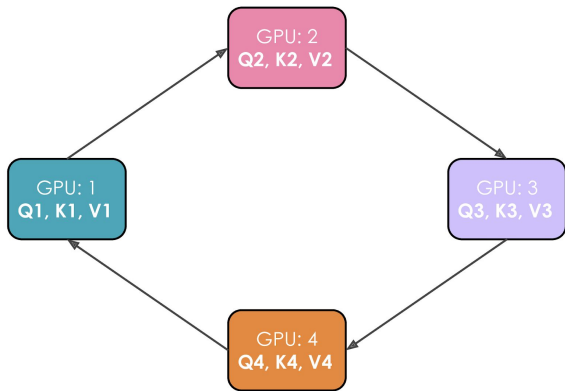
План лекции

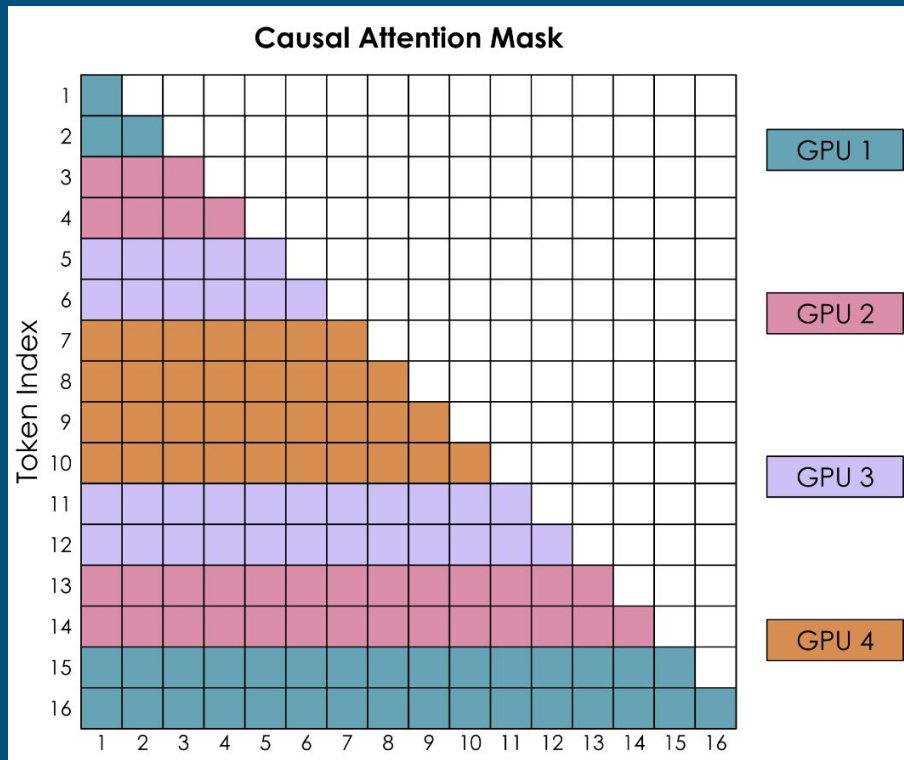
1. Связь нескольких
2. Коммуникации
3. Параллелизм данных (DP)
4. Тензорный параллелизм (TP)
5. **Контекстный параллелизм (CP)**
6. Конвейерный параллелизм (PP)
7. 5D параллелизм

Кольцевое внимание (Ring Attention)



Кольцевое внимание (Ring Attention)





План лекции

1. Связь нескольких
2. Коммуникации
3. Параллелизм данных (DP)
4. Тензорный параллелизм (TP)
5. Контекстный параллелизм (CP)
6. **Конвейерный параллелизм (PP)**
7. 5D параллелизм

AFAB

GPU



Time →

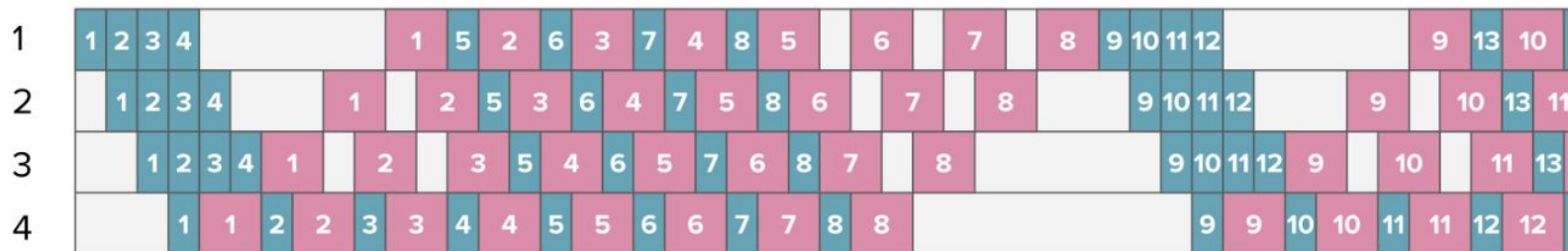
Forward pass

Backward pass

Device idle

1F1B

GPU



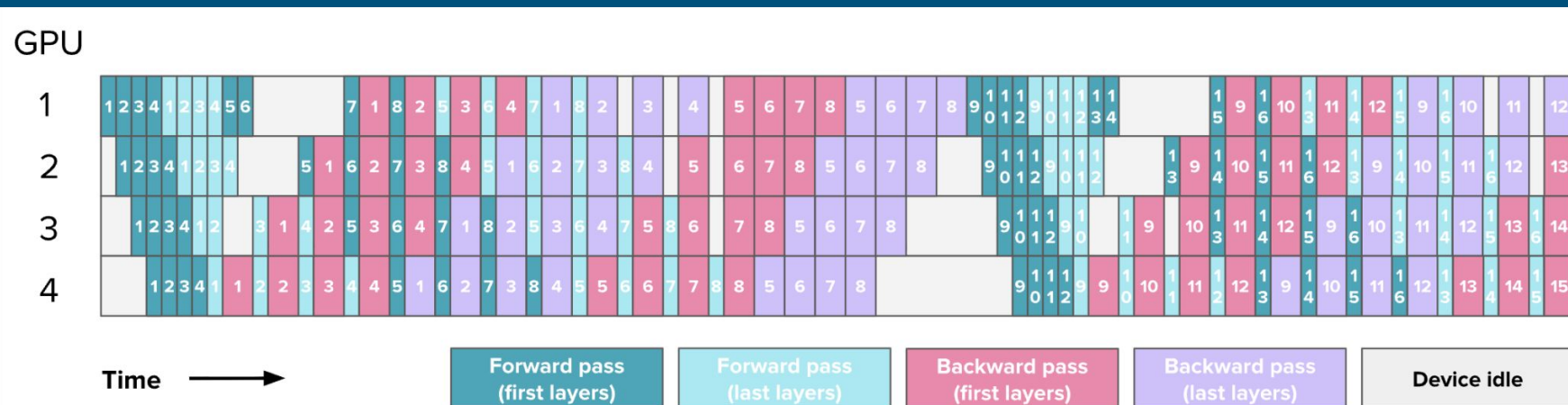
Time →

Forward pass

Backward pass

Device idle

Interleaved PP



План лекции

1. Связь нескольких
2. Коммуникации
3. Параллелизм данных (DP)
4. Тензорный параллелизм (TP)
5. Контекстный параллелизм (CP)
6. Конвейерный параллелизм (PP)
7. **5D параллелизм**

Собираем все параллелизмы вместе

1. TP. Строго внутри ноды, $TP \leq 8$

Собираем все параллелизмы вместе

1. TP. Строго внутри ноды, $TP \leq 8$
2. CP – желательно внутри ноды (но жизнь жестока, часто $CP * TP > 8$)

Собираем все параллелизмы вместе

1. TP. Строго внутри ноды, $TP \leq 8$
2. CP – желательно внутри ноды (но жизнь жестока, часто $CP * TP > 8$)
3. $SP = TP * CP$ (GPU не должны простаивать в ожидании своего часа)

Собираем все параллелизмы вместе

1. TP. Строго внутри ноды, $TP \leq 8$
2. CP – желательно внутри ноды (но жизнь жестока, часто $CP * TP > 8$)
3. $SP = TP * CP$ (GPU не должны простаивать в ожидании своего часа)
4. EP. Вместо FFN мы будем использовать MoE-блоки, но об этом позже.
MoE-блоки можно распараллелить с помощью экспертного параллелизма

Собираем все параллелизмы вместе

1. TP. Строго внутри ноды, $TP \leq 8$
2. CP – желательно внутри ноды (но жизнь жестока, часто $CP * TP > 8$)
3. $SP = TP * CP$ (GPU не должны простаивать в ожидании своего часа)
4. EP. Вместо FFN мы будем использовать MoE-блоки, но об этом позже.
MoE-блоки можно распараллелить с помощью экспертного параллелизма
5. PP – для серьезного масштабирования на медленной межнодовой связи поверх TP/CP/SP/EP

Собираем все параллелизмы вместе

1. TP. Строго внутри ноды, $TP \leq 8$
2. CP – желательно внутри ноды (но жизнь жестока, часто $CP * TP > 8$)
3. $SP = TP * CP$ (GPU не должны простаивать в ожидании своего часа)
4. EP. Вместо FFN мы будем использовать MoE-блоки, но об этом позже.
MoE-блоки можно распараллелить с помощью экспертного параллелизма
5. PP – для серьезного масштабирования на медленной межнодовой связи поверх TP/CP/SP/EP
6. DP – только ZeRO-1 поверх всех остальных параллелизмов.

Вопросы?
